

# Backend Schema and API Contract

---

## Offline Field Issue Tracker

### 1. Relational Schema

The following SQLite DDL is the reference contract. `location` is stored as JSON text to keep the assessment schema portable; production may normalize it or use a spatial type.

```
PRAGMA foreign_keys = ON;

CREATE TABLE reports (
  id TEXT PRIMARY KEY, -- UUID
  category TEXT NOT NULL,
  description TEXT NOT NULL,
  location TEXT NOT NULL, -- JSON: { latitude, longitude, accuracy?, landmark?,
source }
  priority TEXT NOT NULL CHECK (priority IN ('low', 'medium', 'high',
'critical')),
  status TEXT NOT NULL CHECK (
    status IN ('draft', 'submitted', 'assigned', 'in_progress', 'resolved',
'rejected')
  ),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  client_synced_at TEXT
);

CREATE TABLE report_history (
  id TEXT PRIMARY KEY, -- UUID
  report_id TEXT NOT NULL REFERENCES reports(id) ON DELETE CASCADE,
  previous_status TEXT,
  new_status TEXT NOT NULL,
  actor_role TEXT NOT NULL CHECK (actor_role IN ('field_worker', 'coordinator',
'system')),
  note TEXT,
  timestamp TEXT NOT NULL
);

CREATE TABLE sync_logs (
  id TEXT PRIMARY KEY, -- UUID
  idempotency_key TEXT NOT NULL UNIQUE,
  report_id TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('received', 'succeeded', 'failed',
'conflict')),
  error_details TEXT,
  timestamp TEXT NOT NULL
);

CREATE INDEX reports_status_idx ON reports(status);
```

```
CREATE INDEX reports_priority_idx ON reports(priority);
CREATE INDEX reports_category_idx ON reports(category);
CREATE INDEX report_history_report_time_idx
  ON report_history(report_id, timestamp);
CREATE INDEX sync_logs_report_idx ON sync_logs(report_id);
```

## Prisma equivalent

```
enum Priority { low medium high critical }
enum ReportStatus { draft submitted assigned in_progress resolved rejected }
enum ActorRole { field_worker coordinator system }

model Report {
  id          String      @id
  category    String
  description  String
  location    String
  priority    Priority
  status      ReportStatus
  createdAt   DateTime     @map("created_at")
  updatedAt   DateTime     @map("updated_at")
  clientSyncedAt DateTime? @map("client_synced_at")
  history     ReportHistory[]
  @@map("reports")
}

model ReportHistory {
  id          String      @id
  reportId    String      @map("report_id")
  previousStatus ReportStatus? @map("previous_status")
  newStatus   ReportStatus @map("new_status")
  actorRole   ActorRole   @map("actor_role")
  note        String?
  timestamp   DateTime
  report      Report      @relation(fields: [reportId], references: [id],
onDelete: Cascade)
  @@index([reportId, timestamp])
  @@map("report_history")
}

model SyncLog {
  id          String      @id
  idempotencyKey String    @unique @map("idempotency_key")
  reportId    String      @map("report_id")
  status      String
  errorDetails String?    @map("error_details")
  timestamp   DateTime
  @@map("sync_logs")
}
```

## 2. API Conventions

- Base URL: `/api`.
- JSON request and response bodies.
- ISO-8601 UTC timestamps.
- UUIDs are lowercase canonical strings.
- `Idempotency-Key` is required for synchronization.
- Error shape:

```
{
  "error": {
    "code": "INVALID_TRANSITION",
    "message": "Human-readable explanation",
    "fieldErrors": {},
    "requestId": "req_123"
  }
}
```

## 3. Endpoints

### POST `/api/reports/sync`

Synchronizes one or more reports. A batch is bounded (recommended maximum: 50 items) and returns per-item results.

Headers:

```
Content-Type: application/json
Idempotency-Key: sync-command-uuid
```

Request:

```
{
  "items": [
    {
      "id": "0f2b5b62-3b44-4c74-9b0e-3cc6b2fdb1cb",
      "category": "road_damage",
      "description": "Crack across west access road",
      "location": {
        "latitude": 51.501,
        "longitude": -0.141,
        "accuracy": 12,
        "source": "gps"
      },
      "priority": "high",
      "status": "submitted",
      "reported_at": "2026-10-02T10:42:00Z"
    }
  ]
}
```

```
}  
]  
}
```

Response 200:

```
{  
  "results": [  
    {  
      "id": "0f2b5b62-3b44-4c74-9b0e-3cc6b2fdb1cb",  
      "outcome": "synced",  
      "status": "submitted",  
      "serverUpdatedAt": "2026-10-02T10:42:03Z"  
    }  
  ],  
  "requestId": "req_123"  
}
```

Status codes: 200 success/replay, 207 mixed batch outcomes if supported, 400 invalid workflow transition or missing idempotency header, 409 idempotency conflict, 422 validation, 500 server failure.

## GET /api/reports

Query parameters:

```
status=submitted,assigned  
priority=high  
category=road_damage  
createdFrom=2026-10-01T00:00:00Z  
createdTo=2026-10-02T23:59:59Z  
page=1  
pageSize=25
```

Response:

```
{  
  "data": [{ "id": "...", "status": "submitted", "priority": "high" }],  
  "pagination": { "page": 1, "pageSize": 25, "total": 1 }  
}
```

## PATCH /api/reports/:id/status

Request:

```
{
  "newStatus": "rejected",
  "expectedStatus": "submitted",
  "reason": "Duplicate of report 4b0..."
}
```

Returns **200** with the updated report and history entry. Returns **400** for stale/invalid transitions and **422** when a reason or assignee is required but missing.

### GET /api/reports/:id/history

Returns all events in ascending chronological order:

```
{
  "reportId": "...",
  "events": [
    {
      "id": "...",
      "previousStatus": "draft",
      "newStatus": "submitted",
      "actorRole": "field_worker",
      "note": null,
      "timestamp": "2026-10-02T10:42:03Z"
    }
  ]
}
```

## 4. OpenAPI 3.1 Outline

```
openapi: 3.1.0
info:
  title: Offline Field Issue Tracker API
  version: 1.0.0
paths:
  /api/reports/sync:
    post:
      operationId: syncReports
      parameters:
        - in: header
          name: Idempotency-Key
          required: true
          schema: { type: string, minLength: 1 }
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/SyncRequest' }
      responses:
```

```
'200': { description: Synchronized or replayed }
'409': { description: Conflict }
'422': { description: Validation error }
/api/reports:
  get:
    operationId: listReports
    parameters:
      - { in: query, name: status, schema: { type: string } }
      - { in: query, name: priority, schema: { type: string } }
      - { in: query, name: category, schema: { type: string } }
/api/reports/{id}/status:
  patch:
    operationId: transitionReport
    parameters:
      - { in: path, name: id, required: true, schema: { type: string, format:
uuid } }
/api/reports/{id}/history:
  get:
    operationId: getReportHistory
components:
  schemas:
    SyncRequest:
      type: object
      required: [items]
      properties:
        items: { type: array, minItems: 1, maxItems: 50 }
```